

universität wien

BACHELOR'S THESIS

Title of the Bachelor's Thesis

„Developing a ReAct inspired Agent from scratch for utilizing on Large Language Models (LLMs)“

submitted by

Zhaniya Abzhanova

in partial fulfilment of the requirements for the degree of

Bachelor of Science (BSc)

Vienna, 2026

Degree program code:	UA 033 526
Degree program:	Business Informatics
Supervisor:	Dipl.-Ing. Dr.techn. Marian Lux

Abstract

This thesis presents the design and implementation of a from-scratch, local-first ReAct-style agent architecture targeted at open-source LLMs with up to 8B parameters. The proposed system introduces a modular and evaluation-oriented architecture featuring role-based LLM adapters, a configurable action catalogue with Action–Handler separation, deterministic tool input refinement, evidence-first answer synthesis, and confidence-gated retry logic based on modular parameter assessors.

The architecture was implemented as the open-source Python package `my-react-agent` and evaluated against LangChain and LlamaIndex under identical conditions using the same model and toolset. The results indicate that reliability improvements for smaller LLMs can be achieved through architectural control mechanisms rather than model scaling alone. The proposed system helps with building reliable, traceable, and extensible local-first LLM agents.

Contents

Abstract i

List of Tables iv

List of Figures v

List of Algorithms vi

Listings vii

1 Motivation 1
- 1.1 Related Work and Problem Statement . . . 2
- 1.2 Goals and Success Metrics . . . 3
- 1.3 Main contributions . . . 4

2 Design 5
- 2.1 Conceptual Overview . . . 5
- 2.2 Major Design Ideas . . . 5
- 2.3 Why this design is effective . . . 6
- 2.4 Alternative Ideas Considered During Development . . . 7

3 Implementation 8
- 3.1 Code location and Project Structure . . . 8
- 3.2 Prerequisites for Reproduction . . . 8
- 3.3 Main Agent Loop . . . 8
- 3.4 Logging . . . 10
- 3.5 Extendibility and Maintainability . . . 11
- 3.6 Tools Implementation . . . 20
  - 3.6.1 Calculator tool (`CalculatorTool`) . . . 20
  - 3.6.2 Vector search tool (`VectorSearchTool`) . . . 20
  - 3.6.3 Web search tool (`DuckDuckGoTool`) . . . 21
  - 3.6.4 Wikipedia tool (`WikipediaTool`) . . . 21
- 3.7 My Discoveries during Development . . . 21

4 Assessment 23
- 4.1 Benchmark Design . . . 23
- 4.2 Results . . . 23
  - 4.2.1 Correctness . . . 23

4.2.2 Faithfulness . . . 24
4.2.3 Efficiency . . . 25
4.2.4 Robustness . . . 25
4.3 Discussion . . . 25

5 Conclusions and Future Work **26**
5.1 Assumptions and Limitations . . . 27
5.2 Future Work . . . 27
5.2.1 Project-Level Future Work . . . 27
5.2.2 Broader Research Directions . . . 28
5.3 Final Remarks . . . 28

References **29**

List of Tables

4.1 Correctness by query type (successful runs out of 3) 24
4.2 Efficiency summary over all 15 runs per agent 25
4.3 Robustness-focused outcomes 25

List of Figures

3.1 Example of Logging 11
3.2 Component Diagram of the Agent 12

List of Algorithms

1 ReActAgent flow from question to final answer 9

Listings

3.1 Custom AgentTool: MultiplicationTool 14
3.2 Minimal registration 14
3.3 Prompt add + full override 15
3.4 Prompt text-only override 16
3.5 Override of AnswerByItself Action 16
3.6 Custom Action + ActionHandler (short form) 17
3.7 Custom ParameterAssessor integration (short form) 18
3.8 Minimal custom LLMBase adapter 19
3.9 Adapter wiring options 19

1 Motivation

A **Large Language Model (LLM)** is a neural network trained on a very large text dataset to predict the next token (e.g., the next word fragment). Many LLMs are built on the Transformer architecture, which uses an attention mechanism to model relationships between tokens in a sequence [1]. Despite strong capabilities of modern LLMs to analyse vast amounts of information, process it, understand context, and generate answers, they still cannot be called universal assistants. They have limitations. First, the most common problem is that models can produce plausible but incorrect statements, fabricated rather than extracted from the training data, caused by data gaps, probabilistic guessing, or training biases; these are commonly called "hallucinations" [2]. Second, they have difficulties with planning [3]. LLMs are trained to predict the next word rather than optimise for a specific goal or outcome. They may choose a general but incorrect path rather than the correct one for a specific, unique scenario. This can also occur for the first reason (some may hallucinate when performing a complex task). Third, they lack fresh knowledge, since they are trained on data that can become outdated over time [4]. Fourth, they encounter context-window effects, meaning they forget previous information or lose semantic connections due to the volume of information, which makes answers less correct [5]. These problems are not exceptional, as various studies have shown; they are standard limitations because LLMs only generate the next text based on probabilities [3] [6].

But can these limitations be circumvented? In 2023, a study titled **"ReAct: Synergizing Reasoning and Acting in Language Models"** was published. Researchers noted that LLMs possess impressive reasoning abilities, moving from simple pattern matching to multi-step logical inference, mathematical computation, and programming, particularly when using methods such as chain-of-thought (CoT) prompting, as well as action, namely, step-by-step decision-making processes [7]. Thus, models can be augmented with tools by structuring interactions through an iterative cycle of reasoning and action. The key idea is to think about what information is missing, act to obtain it (e.g., search/retrieve/calculate), and then use the observation to continue. Does this fix the model? The answer is no, but it reduces the impact of its failures:

- **Relevant information**: If a question depends on recent facts, a ReAct agent can use web search or other real-time tools rather than providing answers based on knowledge acquired during training [4]. For example, systems like WebGPT show that browsing and citing sources can improve answers to factual questions [8].
- **Fewer hallucinations**: When the agent retrieves data, the answer is tied to real sources rather than invented by the model.

- **Handling long documents**: Instead of inserting a huge document into the prompt (which can cause important information to be missed) [5], the agent can use vector search tool to get the most relevant fragments. This improves accuracy and reduces noise [9].
- **Accurate computation**: For math, the agent invokes a calculator or code rather than trying to calculate "in its head" using text probabilities what reduces arithmetic errors.

1.1 Related Work and Problem Statement

The ReAct agent has expanded the capabilities of LLMs and from a research idea has become a widely used framework paradigm. The most popular open sourced frameworks are Llama index [10], Langchain [11], Dynamiq [12] and AutoGen [13]. For this thesis as baselines are chosen LangChain and LlamaIndex because of their reliability and functionality. But where do the differences between them lie?

LangChain

In LangChain, agent execution is managed as a graph (LangGraph) rather than a single linear chain [14]. That means that the agent sends messages containing instructions and tool schemas to the model. If the model returns a tool call, the agent executes the tool and stores the output as a tool message. The agent then calls the model again with the updated history. This process repeats until no further tool calls are produced. Around this loop, LangChain also provides practical controls, such as middleware hooks, dynamic tool and model selection, structured output strategies, and persistence through stores.

LlamaIndex

LlamaIndex executes the same general loop: the agent reasons step by step, selects tools, and forms a final response based on tool outputs. However, the main difference and advantage of this framework is compatibility. The agent can operate even when the LLM does not support native function calling. This makes the LlamaIndex ReAct agent more robust and less dependent on a specific tool API. LlamaIndex agent's loop is events during execution, and allows inspection of artifacts such as incremental agent outputs and tool-invocation results [15].

Why related frameworks don't fully solve the problem

The documentation of popular agent frameworks makes building a ReAct-style system look easy. In practice, developers often run into the same set of problems. Recent empirical work on defects in LLM applications reports that frameworks such as LangChain and LlamaIndex frequently break in subtle ways, and that small changes in prompts or configuration can lead to failures that are hard to reproduce and debug [16].

One issue is that these frameworks offer many architectural options from the start: different agent abstractions, different ways to format tools, multiple memory patterns, and different streaming setups. This flexibility is useful, but it also means developers can spend a lot of time choosing an architecture before they even have a stable prototype. This turns into an engineering challenge. The system needs clear boundaries and measurable behaviour, not just many configuration choices [17].

Another practical problem is tool-call fragility. If the tool description is vague, the schema is strict, or the model hallucinates a tool name, the step can fail or the agent can get stuck in a loop. This is not only a reliability issue; research also shows that tool-mediated LLM applications can be pushed into unsafe or unintended actions through prompt-level manipulation [18].

Debugging is closely connected to these issues. When something goes wrong, it is often unclear whether the root cause is the prompt, the tool schema, how messages are ordered, what was stored in memory, or the model itself. Without a detailed step trace, finding the problem can take a long time [16].

Therefore, building the agent core from scratch is not about reinventing LangChain or LlamaIndex. The limitations identified above motivate the specific improvements targeted in this thesis and form the foundation for the goals and success metrics described in the next section.

1.2 Goals and Success Metrics

Goals

This thesis aims to make the control loop more transparent, testable, and measurable, and to demonstrate that the proposed architecture improves reliability, usability and extendibility specifically in the constrained setting of local open-source models (with maximum 8B datasets).To support this aim, the agent implements mechanisms that address the issues observed in widely used frameworks: question splitting, strict tool-input refinement, confidence-based recovery, and evidence-first answer synthesis.

The first mechanism is question splitting, which reduces planning complexity by breaking complex queries into smaller sub-tasks and makes failures traceable to specific steps rather than appearing only as vague issues in the final answer. The second mechanism is strict tool-input refinement, which improves reliability by rewriting and validating tool arguments according to tool-specific refinement instructions before execution; this prevents malformed calls and makes tool interactions more deterministic. The third is confidence-based recovery that evaluates each step's output quality and triggers controlled retries—such as re-planning, refining inputs, or switching tools—when confidence is low, preventing single mistakes from derailing the entire run. Finally, evidence-first answer synthesis grounds the final response in tool outputs and retrieved data, which reduces hallucinations, encourages explicit uncertainty when evidence is missing, and makes the output auditable.

Metrics

The proposed agent will not be assessed in isolation. Instead, it evaluates whether a from-scratch, local-first ReAct-style agent can match or even outperform widely used agent frameworks - LangChain and LlamaIndex - on the same query set under controlled conditions. The evaluation is based on four metrics: correctness, faithfulness, efficiency and robustness. Correctness captures whether the final response is factually and logically valid, consistent with standard used in Question Answering (QA) and Large Language (LM) benchmarks [19, 20]. Faithfulness measures whether answers are grounded in the retrieved evidence rather than generated from unsupported assumptions, which is a common requirement in grounded generation and RAG evaluation [21, 22]. Efficiency evaluates how quickly and economically the agent reaches a correct answer, a dimension also considered in holistic LM evaluation [20]. Robustness measures stability under realistic variation in user input and under partial system failures, aligned with behavioral and robustness testing approaches for NLP systems [23, 24].

1.3 Main contributions

The main contribution of this thesis is the explicit agent workflow: a structured plan-and-execute control loop designed for local open-source LLMs. A second contribution is a modular architecture built around stable contracts, so new tools, prompts, or model backends can be added by plugging in modules rather than rewriting the core loop. In addition, the system records plans, tool evidence, confidence decisions, and synthesis outputs in structured memory and logs, making the workflow observable and reproducible.

Finally, a practical outcome of these contributions is the agent's usability for developers. The thesis provides a ReAct-inspired agent that is lightweight, easily extensible, and maintainable, and it is released as a PyPI package (`my-react-agent`) that makes it an open-source framework.

2 Design

This chapter translates the goals and contributions described in the Chapter 1 into a concrete system design.

2.1 Conceptual Overview

The implemented system is a **ReAct agent built from scratch**. The agent starts with a decision whether the user query should be decomposed into sub-tasks or not. After the agent creates a plan with one or more steps, depending on query decomposition, to answer the query. Then each step of the plan is solved with the most suitable action (answer directly, call a tool, ask clarification, resolve context or stop). If the model answers the step's task byitself or uses tool to get external information, it is assessed as the next part of the loop.If the assessment is low the step will be repeated. This design follows the core ReAct idea [7], but is adapted for local open-source LLMs (with fewer than 8B parameters) running through Ollama [25]. As mentioned in the first chapter, the goal was to make the framework modular and maintainable. Therefore, the architecture to implement these iterations is built around clean interfaces (LLM adapters, tools, actions, handlers, assessors), so components can be added or swapped without changing the core execution loop.

2.2 Major Design Ideas

One of the significant differences between the proposed architecture and open-source ReAct frameworks is the existence of **pluggable LLM adapters**. The agent can assign different Ollama models to specific roles: the **planner llm** decides decomposition and per-step action, the **summariser llm** builds step summaries and the final answer, the **refiner llm** rewrites tool inputs with the help of tool-refiner instructions defined in tool classes, and the **confidence llm** supports confidence assessment of model answers or tool results. This design decision makes the system flexible: the same model can be used for all roles, or different models can be mixed and matched per role. For example, a stronger model can be assigned as planner and a cheaper model as refiner.

The next essential design idea is innovative is a customisable action catalogue, where actions are the agent's explicit "next-step decisions". The action catalogue includes default actions (USE TOOL, ANSWER BY ITSELF, CLARIFY, RESOLVE PRONOUNS AND OMITTED ENTITIES, and STOP) and can be easily extended with new actions. **USE TOOL** Action is selected when the step depends on external or time-sensitive information and an available tool can solve it with one deterministic call. **ANSWER BY ITSELF**

is used only when the step can be answered from stable, textbook-level knowledge or when the answer is already known from a previous step, so no tool call is needed. When the user request is underspecified, the agent picks **CLARIFY** and asks a single short clarification question with a few inline options instead of guessing. If the step is unclear because it contains pronouns or omitted entities, **RESOLVE PRONOUNS AND OMITTED ENTITIES** rewrites the step into an explicit, self-contained instruction by replacing those references with concrete entities from context. Finally, **STOP** ends the session repeated failures indicate that continuing would likely lead to looping rather than progress.

The architecture adopts **Action-Handler separation**. This means that the agent explicitly separates *what to do* from *how it is executed*. In addition to actions, there are handlers that implement runtime logic. This allows custom behavior (e.g., new domain actions) by registering a new action-handler pair without editing the core codebase. As a result, the approach provides a practical alternative to monolithic agent pipelines and directly supports the extensibility expected in framework-like systems.

The agent is **tool-centric**. That is why the quality of tool results is significant. For this reason, a refinement process is applied before tool calls. The flow is: the agent chooses the tool at planning time, refines input to a tool-specific format (described in tool classes), and executes the tool. This is useful for heterogeneous tools (vector search, web, Wikipedia, calculator), which have different requirements for their input contracts. Finally, the agent converts tool output into structured evidence objects in a common format used across the system.

The architecture is **memory-aware** and supports context repair through two memory layers. **Query memory** caches current run status, plan, transcript, and step history. On a higher level, **Conversation memory** stores recent question-answer pairs and entities extracted from step results. Because local LLMs have limited context windows, the agent uses a sliding-window strategy for conversation memory (last 5 turns as default but can be changed) and constructs prompts from compact subsets such as the most recent step outputs, extracted entities/facts, and short evidence snippets.

To prevent errors from propagating across steps, the agent applies a **confidence-gated retry loop** after each step. Result quality is scored using parameter assessors (default assessors are alignment, quality, and realism). If confidence is low, the agent enters a **low-confidence recovery policy** that re-decides the next action and retries with constraints (for example, it avoids repeating the same failed tool/input pairs). This assessment is crucial for small local models because it helps compensate for weaker raw generation quality.

2.3 Why this design is effective

Subsequently, this design is effective under the project's defined constraint: small local LLMs. The architecture provides robustness without requiring very large models by separating model roles and having confidence-gated recovery. Furthermore, it fullfils the main contribution defined in the first chapter: the architecture was intentionally designed

for high extensibility and maintainability. The system is highly extensible, since new tools, actions, assessors, and LLM backends can be integrated with minimal code changes through modular interfaces and the action–handler structure.

From engineering perspective, the design is production-oriented. Explicit memory layers, trace recording, standardised initialisation, and clear handler boundaries make the codebase easier to maintain and evolve. These properties support open-source development and practical packaging and distribution (e.g., for PyPI-style workflows). Finally, the usage pattern is similar to modern agent frameworks, making the system familiar to developers, while the implementation itself remains transparent and fully built from scratch.

2.4 Alternative Ideas Considered During Development

The main ideas described in this chapter did not emerge as the only option. Several alternative designs were explored during development, but they produced weak results and were not included in the final architecture:

1. **One model for every role.** In the beginning one LLM was used for planning, tool formatting, summarisation, and confidence checks. It was easy to run, but hard to tune: improving one part often made another part worse.

2. **Single monolithic loop (no action catalogue).** Early versions used big `if/else` blocks inside the main loop. That worked for quick prototypes, but became messy and fragile once more actions were added.

3. **Putting execution code inside Action classes (no handlers).** This mixed "what to choose" with "how to execute it," which made the code harder to read and test. Splitting actions from handlers gave cleaner responsibilities.

4. **Raw natural-language tool calls (no refiner).** Letting the model send tool input directly seemed convenient at first, but caused many malformed inputs and unstable tool behavior. Adding a refiner made tool calls much more consistent.

5. **One global confidence prompt (no modular assessors).** A single confidence prompt was simple, but too opaque. When confidence dropped, it was unclear why. Modular assessors made confidence scoring easier to debug and extend.

6. **Prompts hardcoded in classes.** Hardcoding prompts slowed down iteration because every small prompt tweak required code edits. Moving prompts to a registry made experiments, overrides, and versioning much easier.

3 Implementation

This part of thesis turns the design described in the previous chapter into a concrete implementation of the proposed ReAct agent. The goal is to show how exactly was reliability, traceability, and maintainability achieved in practice, and to provide all information needed to reproduce the work.

3.1 Code location and Project Structure

The project is distributed in two places. For installation and quick reuse, it is published as the **my-react-agent** package on PyPI. For full source access, version history, and reproducibility, the complete codebase is available in the project Git repository.

The implementation is organised into clear modules. The core orchestration loop is implemented in `my_react_agent/agent_heart/react_agent.py`, while the action catalogue is defined in `my_react_agent/agent_core/agent_actions/` and executed through runtime handlers in `my_react_agent/agent_heart/react_handlers/`. Tool invocation and input refinement are handled in `my_react_agent/tool_management/`, confidence scoring logic is in `my_react_agent/confidence_assessment/`, and memory components for queries and conversations are in `my_react_agent/agent_memory/`. Prompt definitions and defaults are centralised in `my_react_agent/agent_prompts/`. In addition, the `evaluation/` directory contains implemented tools and execution scripts used for reproducible comparison between the proposed agent, LangChain, and LlamaIndex.

3.2 Prerequisites for Reproduction

To reproduce the prototype and demo setup, the runtime environment needs to satisfy a small set of practical requirements. The project runs on Python 3.11 or newer [26], and Ollama must be installed and running locally [25]. At least one local model has to be pulled in advance, since the agent cannot execute planning or synthesis without an available backend model. In addition, startup validation requires at least one registered tool. If no tool is configured, the agent initialisation is rejected by design.

3.3 Main Agent Loop

As shown in **Algorithm 1**, execution starts by initialising a new query session and transcript (lines 1–2), then decides whether the question should be decomposed and builds the plan accordingly (lines 3–5). The plan is stored in QueryMemory and an empty execution trace is created (lines 6–7). The agent then iterates over the planned steps up

```
(1) Start query session; set status RUNNING; reset per-run state
(2) transcript ← [user question]
(3) if ShouldSeparateQuestion(q) then
  | (4) plans ← PlanQuestionSteps(q)
else
  | (5) plans ← [single-step plan from q]
end
(6) Save plans to QueryMemory
(7) executed_trace ← [ ]
(8) foreach step in plan up to max_steps do
  | (9) decision ← DecideActionForStep(q, plan)
  | (10) retries ← 0
  | (11) while true do
  |   | (12) Run action handler for decision.action and get step_result
  |   | (13) if decision.action = NEED_CONTEXT then
  |   |   | (14) optionally rewrite step instruction
  |   |   | (15) re-decide action for the same step
  |   |   | (16) continue
  |   | end
  |   | (17) Build step summary evidence
  |   | (18) if step should be confidence-assessed then
  |   |   | (19) compute confidence conf
  |   | end
  |   | (20) if conf < threshold and retries < max_step_retries then
  |   |   | (21) decision ← low-confidence recovery decision
  |   |   | (22) retries ← retries + 1
  |   |   | (23) continue
  |   | end
  |   | (24) if conf < threshold and retries exhausted then
  |   |   | (25) store failure message for this step
  |   | end
  |   | else
  |   |   | (26) persist step trace/evidence to memory
  |   |   | (27) append successful step to executed_trace
  |   | end
  |   | (28) if step_result.should_stop then
  |   |   | (29) finalise memory with current answer
  |   |   | (30) return transcript
  |   | end
  |   | (31) break
  | end
end
(32) final ← SynthesiseFinalAnswer(q, executed_trace, context)
(33) append final to transcript; persist transcript and memory
(34) return transcript
```

Algorithm 1: ReActAgent flow from question to final answer

to `max_steps` (line 8). For each step, it selects an action and initialises a bounded retry counter (lines 9–10), and executes the chosen action through its handler inside a retry loop (lines 11–12). If the action is `NEED_CONTEXT`, the step instruction can be rewritten and the action is re-decided for the same step without advancing the plan index (lines 13–16). For regular steps, the agent builds step evidence and, when enabled, computes a confidence score (lines 17–19). If confidence is below the threshold and retries remain, the agent enters low-confidence recovery by choosing a revised decision and re-running the step (lines 20–23). If retries are exhausted, the step is marked as failed; otherwise, successful evidence is persisted and appended to `executed_trace` (lines 24–27). The loop can terminate early if `step_result.should_stop` is true, in which case memory is finalised and the transcript is returned immediately (lines 28–30). The retry loop then exits and execution proceeds to the next plan step (line 31). Finally, after all steps, the agent synthesises the final answer from the executed trace, persists the run state, and returns the transcript (lines 32–34).

3.4 Logging

Important part of implementation is structured runtime logging because it makes the agent behaviour auditable. Each line follows the format:

`timestamp level [module] [function] message`

This format makes it possible to trace *what happened*, *where in code it happened*, and *in what order*.

Logging example on figure 3.1 represents a complete execution trace for the query "round the number 66663.432424234 to 2 decimal place". The run begins with standard initialization: the agent opens a new query session and links it to conversation memory (`New query session started`, `ConversationMemory linked`). During planning, the model decides that decomposition is unnecessary (`separate=False`), so the task is processed as a single-step plan. For Step 1, the selected action is `ANSWER_BY_ITSELF`, and the step is therefore executed directly without invoking any external tool. After execution, the confidence pipeline evaluates the result using `EntityAlignmentAssessor`, `AnswerQualityAssessor`, and `AnswerRealismAssessor`. All three assessors return a score of 1.00, which is above the configured threshold (`conf=1.00 >= threshold=0.70 -> ok`), so no retry is performed. The agent then persists the step output, transcript, and conversation turn in memory. The run terminates successfully with `executed_steps=1`, and the final answer is 66663.43.

This logging design is useful for three reasons:

1. Logging improves debugging: when a run fails, the trace makes it clear whether the issue comes from planning, action selection, tool execution, confidence gating, or memory updates.



Figure 3.1: Example of Logging

2. Logging supports reproducibility: query identifiers, step counters, thresholds, and assessor outputs create an exact execution record that can be rerun and compared across experiments.

3. Logging strengthens evaluation: the logs expose the full decision path and confidence signals behind each final answer, making the agent's behaviour transparent.

Practical note. `INFO` is used for high-level flow and `DEBUG` for detailed internals (prompt lengths, cache updates, assessor internals). This gives a good balance between readability and depth by debugging.

3.5 Extendibility and Maintainability

Figure 3.2 demonstrates how the implementation is split into a set of subsystems with clear responsibilities. And it directly supports the goals stated in the first chapter: the

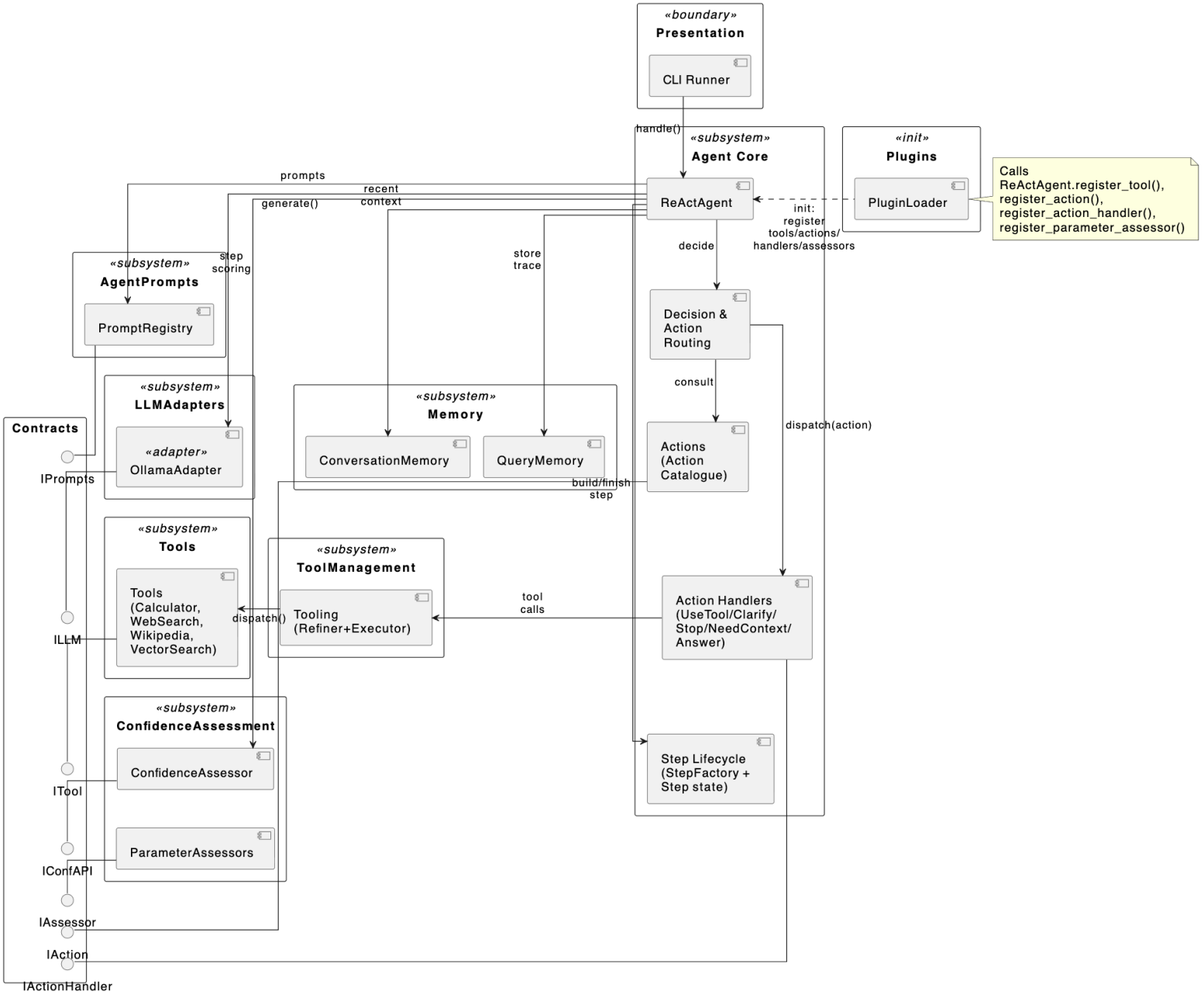


Figure 3.2: Component Diagram of the Agent

agent is practical to use and easy to extend.

Why the architecture is extendable and maintainable

Although Python does not provide interfaces as a built-in language feature, the implementation follows an *interface-like* approach by relying on clear **contracts**. In Figure 3.2, these contracts appear as `ILLM`, `IPrompts`, `IConfAPI`, `IParameterAssessor`, and `IAgentTool`; in the code they correspond to extension bases such as `LLMBase`, `PromptRegistry`, `ConfidenceAssessor`, `ParameterAssessor`, and `AgentTool`. This makes extensibility a first-class property: tools plug in through `AgentTool` and run through `ToolExecutor` (see `Tools` → `ToolManagement` in Figure 3.2), prompts are consistently accessed via `PromptRegistry`, confidence logic is concentrated in `ConfidenceAssessor` and extended by adding new `ParameterAssessor` modules, and model backends remain replaceable behind the Ollama adapter in `LLMAdapters`.

The system is also designed around **dependency injection**, where the agent is assembled by passing dependencies into `ReActAgent.create(...)` (or directly into the constructor): tools, the prompt registry, the action catalogue and handlers, parameter assessors, and LLM adapters. This startup wiring is reflected by the `Plugins` subsystem in Figure 3.2. Because the main loop depends on contracts rather than concrete implementations, switching an LLM backend, adding a tool, introducing a new assessor, or overriding a prompt becomes just a configuration change.

Maintainability is further supported by clear **subsystem boundaries** with low coupling and small public surfaces. `ToolManagement` owns tool execution and input refinement (`ToolExecutor`, `ToolQueryRefiner`); `AgentPrompts` owns prompt templates and defaults (`PromptRegistry`, `DEFAULT_PROMPTS`); `ConfidenceAssessment` owns scoring and retry signals (`ConfidenceAssessor` plus parameter assessors); and `LLMAdapters` encapsulate provider-specific calls (e.g., Ollama). This separation reduces regression risk, since improving prompts or adding a tool does not require modifying unrelated modules such as memory or confidence logic.

Finally, the architecture is **traceable by default**, which is essential for evaluation and debugging. The agent records plans and step traces in `QueryMemory` and stores short cross-turn state in `ConversationMemory` (both under `Memory` in Figure 3.2). Together with structured logging and the `Evidence` objects produced by tools (via `ToolManagement` and `Tools` in Figure 3.2), each run produces an auditable record. This makes step counts, tool usage, retries, and confidence decisions measurable artefacts.

Five practical extension patterns

1) Add a new tool (capability extension). Adding a tool is intentionally a small, self-contained change. You implement the `AgentTool` interface and then register the instance when constructing the agent. In practice, this means the tool has its own inputs, validation rules, and execution logic, while the agent core only sees a stable contract: `name`, `description`, optional refiner hints (format/instructions/regex), and `execute(input) -> Evidence`. This keeps the main loop unchanged and makes each tool easy to test in

isolation.

Listings 3.1 and 3.2 illustrate the workflow in practise. `MultiplicationTool` defines a strict input format ("`<a>*<b>`"), provides a regex to catch malformed calls early, and returns a structured `Evidence` object with both a human-readable result and machine-readable fields. Once implemented, the tool becomes available to the agent by simply passing it in the `tools=[...]` list at creation time.

Minimal steps:

1. Implement Subclass `AgentTool` with `name`, `description`, refiner hints (optional but recommended), and `execute()` methods.

2. Register the tool by passing an instance into `ReActAgent.create(..., tools=[...])`.

```python
from datetime import datetime, timezone
from typing import Optional
from my_react_agent.agent_memory.data_structures import Evidence
from my_react_agent.tool_management.tools.agent_tool import AgentTool

class MultiplicationTool(AgentTool):
    @property
    def name(self) -> str: return "multiplier"
    @property
    def description(self) -> str: return "Multiply two numbers: '<a>*<b>'."
    @property
    def refiner_instructions(self) -> str: return "Output ONLY '<a>*<b>'."
    @property
    def refiner_input_format(self) -> str: return "<number>*<number>"
    @property
    def refiner_input_regex(self) -> Optional[str]:
        return r"^\s*-?\d+(\.\d+)?\s*\*\s*-?\d+(\.\d+)?\s*$"

    def execute(self, input: str) -> Evidence:
        a, b = input.split("*", 1)
        result = float(a) * float(b)
        return Evidence(
            tool=self.name,
            content=f"{a.strip()} * {b.strip()} = {result}",
            extracted={"expression": input, "result": result},
            as_of=datetime.now(timezone.utc),
            confidence=0.95,
        )
```

Listing 3.1: Custom AgentTool: MultiplicationTool

```python
from my_react_agent.agent_heart.react_agent import ReActAgent
agent = ReActAgent.create(llm="llama3:8b", tools=[MultiplicationTool()])
print(agent.handle("What is 7*8?"))
```

Listing 3.2: Minimal registration

2) Prompting system: add or override prompts. Instead of hardcoding prompt strings across the codebase, the agent reads everything through a centralised layer: `PromptRegistry` stores templates (`PromptTemplate`) and exposes them via stable identifiers (`PromptId`). The default behavior comes from `DEFAULT_PROMPTS`, but any prompt can be extended or replaced without touching the orchestration logic in `ReActAgent`.

Listings 3.3 and 3.4 show the two common workflows. First, a new prompt can be registered under a custom ID, which is useful for experiments or adding new capabilities. Second, an existing built-in prompt can be overridden either by providing a full `PromptTemplate` (including metadata like required variables and a version tag) or by replacing only its text with `set_text(...)` when you want a lightweight tweak.

Minimal steps:

1. Create (or reuse) a `PromptRegistry` seeded with defaults, e.g.

   `PromptRegistry(_defaults=dict(DEFAULT_PROMPTS))`.

2. Add a new prompt via `register_default(...)` or override an existing one via `set(...)` / `set_text(...)`.

3. Pass the registry into the agent with `ReActAgent.create(..., prompts=prompts)`.

```python
from my_react_agent.agent_prompts.prompt_registry import PromptRegistry
from my_react_agent.agent_prompts.prompt_template import PromptTemplate
from my_react_agent.agent_prompts.defaults_prompts import DEFAULT_PROMPTS
from my_react_agent.agent_prompts.prompts_ids import PromptId

prompts = PromptRegistry(_defaults=dict(DEFAULT_PROMPTS))

# Add new prompt
prompts.register_default(
    "my_custom_prompt_id",
    PromptTemplate(
        text="Question:\n{question}\nReturn JSON only.",
        required_vars={"question"},
        description="Custom prompt",
        version="1",
    ),
)

# Override built-in prompt template
prompts.set(
    PromptId.SHOULD_SEPARATE_QUESTION,
    PromptTemplate(
        text="User question:\n{question}\nSchema:\n{schema}\nJSON only:",
        required_vars={"question", "schema"},
        description="override",
        version="override-1",
    ),
)
```

Listing 3.3: Prompt add + full override

```
prompts.set_text(
    PromptId.SHOULD_SEPARATE_QUESTION,
    "User question:\n{question}\nSchema:\n{schema}\nJSON only:"
)
```

Listing 3.4: Prompt text-only override

3) Add or override actions (decision policy extension). Overriding an existing action.

In some cases it's easier to change an existing action rather than to implement a new action from scratch. In practice, this means modifying fields such as `when_to_pick` (when the planner should choose this action) or `instructions` (how the model should execute it). This approach keeps behaviour changes local and avoids duplicating runtime logic.

Listing 3.5 shows an example of overriding the default `AnswerByItselfAction`. It is reused, but its routing hint is tightened so the planner prefers it for conceptual questions and avoids using it for time-sensitive data. The customised action is then passed into the agent through the `step_actions=[...]` list, which replaces the default action set for planning.

So, minimal steps for overriding an action are:

1. Instantiate the built-in action you want to customise.
2. Adjust `when_to_pick` and/or `instructions` to match a desired behaviour.
3. Provide the customised instance via `step_actions=[...]` when creating the agent.

```
from my_react_agent.agent_core.agent_actions import AnswerByItselfAction
from my_react_agent.agent_core.agent_actions import ClarifyAction,
    UseToolAction, StopAction
from my_react_agent.agent_core.agent_actions.need_context_action import
    NeedContextAction

answer = AnswerByItselfAction()
answer.when_to_pick = (
    "Prefer for timeless conceptual questions; avoid for live/current
    data."
)

step_actions = [NeedContextAction(), answer, ClarifyAction(),
    UseToolAction(), StopAction()]
```

Listing 3.5: Override of AnswerByItself Action

Adding a completely new action.

When the existing action set is not sufficient, a new action can be introduced by defining both the planner-facing definition and the runtime execution logic. The architecture intentionally separates these two concerns.

Listing 3.6 shows a minimal example. The `StarWarsAnswerAction` exposes metadata used by the planner (name, routing hints, default instructions). The matching `Star Wars`

`Answer Handler` implements the runtime behaviour and must use the same action key. The handler is responsible for building the step result, attaching evidence if needed, and deciding whether execution should stop.

To activate the new action, it must be registered in two places: in the planner-visible action lists (`step_actions` and optionally `low_conf_actions`), and in the runtime handler mapping (`action_handlers={"KEY": handler}`).

Minimal steps:

1. Implement a new `Action` subclass with `name`, `default_when_to_pick`, and `default_instructions`.
2. Implement a matching `ActionHandler` using the same action name and providing the runtime `run(...)` logic.
3. Register the action in the planner action lists and register the handler in the `action_handlers` mapping.

```python
from my_react_agent.agent_core.agent_actions.action import Action
from my_react_agent.agent_heart.action_handler_base import ActionHandler

class StarWarsAnswerAction(Action):
    @property
    def name(self) -> str: return "STARWARS_ANSWER"
    @property
    def default_when_to_pick(self) -> str: return "Pick for Star Wars questions."
    @property
    def default_instructions(self) -> str: return "Answer with 'May the Force be with you!'."

class StarWarsAnswerHandler(ActionHandler):
    @property
    def action_name(self) -> str: return "STARWARS_ANSWER"
    def run(self, ctx):
        # build final response, set should_stop=True, attach evidence, return updated step
        ...
```

Listing 3.6: Custom Action + ActionHandler (short form)

4) Extend reliability: custom `ParameterAssessor`. The confidence layer is designed to be extensible: instead of relying on one monolithic "judge prompt", the agent can run multiple small assessors, each scoring one concrete quality dimension. When a new reliability criterion is needed (for example, stricter math checking), it can be added without touching the orchestration logic in `ConfidenceAssessor`.

Listing 3.7 shows a minimal assessor implementation. `MathExpressionAssessor` subclasses `ParameterAssessor` and defines a stable identifier (`name`), the prompt it uses (`prompt_id`), and the actual scoring routine in `assess(...)`. Internally, `assess(...)`

renders the assessor prompt, calls the LLM, and converts the response into a normalised `ParameterRating` (score in $[0, 1]$ plus a short reason). Because assessors are registered as factories, they can be wired into the agent at creation time and receive the same `llm` and `PromptRegistry` dependencies as the rest of the system.

In practice, adding an assessor has two parts: provide the prompt template (registered in `PromptRegistry`) and provide a factory function that constructs the assessor. Once the factory is passed via `parameter_assessors=[...]`, the confidence pipeline automatically includes the new score when computing step confidence and triggering retries.

How to add an assessor:

1. Implement a new `ParameterAssessor` subclass (define `name`, `prompt_id`, and `assess(...)`).
2. Register the assessor prompt template in `PromptRegistry` under the same `prompt_id`.
3. Provide a small factory function and pass it into `ReActAgent.create(...)` via `parameter_assessors=[...]`.

```python
from my_react_agent.confidence_assessment.parameter_assessor import ParameterAssessor
from my_react_agent.confidence_assessment.models import ParameterRating

class MathExpressionAssessor(ParameterAssessor):
    @property
    def name(self) -> str: return "math_expression_correctness"
    @property
    def prompt_id(self) -> str: return "confidence_math_expression_min"
    def default_prompt_template(self): return CONF_MATH_EXPR_TEMPLATE

    def assess(self, *, query_text: str, answer_text: str, **_) -> ParameterRating:
        raw = self.llm.generate(self._render_prompt(question=query_text, answer=answer_text))
        score = ...  # parse score from JSON, clamp to [0,1]
        return ParameterRating(name=self.name, score=score, reason="math check")

prompts.register_default("confidence_math_expression_min", CONF_MATH_EXPR_TEMPLATE)

def math_assessor_factory(llm, prompts):
    return MathExpressionAssessor(llm=llm, prompts=prompts)

agent = ReActAgent.create(
    llm="llama3:8b",
    tools=[MultiplicationTool()],
    prompts=prompts,
    parameter_assessors=[math_assessor_factory],
)
```

Listing 3.7: Custom ParameterAssessor integration (short form)

5) Extend model backends: LLM adapter extension. To keep the agent portable, all model providers are hidden behind a single, small contract: `LLMBase.generate(prompt, **kwargs) -> str`. The core loop never talks to Ollama (or any other backend) directly—it only depends on `ILLM/LLMBase`. This means switching models or providers is mostly wiring: you swap an adapter or change configuration, rather than rewriting agent logic.

Listing 3.8 shows a minimal backend adapter for Ollama. The adapter stores backend-specific settings (model name, temperature) and implements `generate(...)` by calling `ollama.chat`. Any backend failures are converted into a clear exception, so errors surface at the right abstraction level and are easy to debug. Once an adapter exists, it can be injected into `ReActAgent.create(...)` either as a single instance for all roles or as a per-role mapping (Listing 3.9). This is particularly useful in the local-first setting, where different roles benefit from different trade-offs (e.g., a stronger model for planning and a smaller model for refinement or confidence checks).

How to add a new LLM adapter:

1. Subclass `LLMBase` (or implement the same `generate(...)` interface).
2. Implement `generate(prompt, **kwargs) -> str`, and wrap backend errors with informative exceptions.
3. Pass the adapter into `ReActAgent.create(...)` either as one shared instance or as a role mapping, depending on how much control you need.

```python
import ollama
from my_react_agent.llm_adapters.llm_base import LLMBase

class OllamaDeepseekLLM(LLMBase):
    def __init__(self, model: str = "deepseek-r1:8b", temperature: float = 0.1):
        self.model = model
        self.temperature = temperature

    def generate(self, prompt: str, **kwargs) -> str:
        try:
            r = ollama.chat(
                model=self.model,
                messages=[{"role": "user", "content": prompt}],
                options={"temperature": kwargs.get("temperature", self.temperature)},
            )
            return r["message"]["content"]
        except Exception as e:
            raise ConnectionError(f"Ollama call failed for model={self.model}: {e}") from e
```

Listing 3.8: Minimal custom LLMBase adapter

```python
# A) single model string
agent = ReActAgent.create(llm="llama3:8b", tools=tools)
```

```
# B) one adapter for all roles
agent = ReActAgent.create(llm=OllamaDeepseekLLM("deepseek-r1:8b"), tools=
    tools)
# C) role mapping
agent = ReActAgent.create(
    llm={
        "default": "llama3:8b",
        "planner_llm": "deepseek-r1:8b",
        "summariser_llm": "deepseek-r1:8b",
        "confidence_llm": "qwen2.5:3b",
        "refiner_llm": "qwen2.5:3b",
    },
    tools=tools,
)
```

Listing 3.9: Adapter wiring options

3.6 Tools Implementation

As already mentioned before, tools are treated as interchangeable components. They implement the `AgentTool` contract: a stable identifier (`name`), a short capability description (`description`), and a single entry point `execute(input) -> Evidence`, optional input-refinement hints (expected format, validation regex, forbidden patterns, examples).The agent interacts with all tools through this interface, keeping planning, execution, and confidence logic consistent across all tool types.

3.6.1 Calculator tool (`CalculatorTool`)

The calculator tool is designed for safe, reproducible arithmetic. It parses expressions with Python's AST and evaluates only a restricted set of operators and functions (e.g., `+,-,*,/,**,round` and a limited `math.*` subset), preventing arbitrary code execution. It also validates inputs (e.g., rejects empty or overly long expressions) and normalises numeric output so rounding is consistent across runs. On failure, it returns structured error evidence with reduced confidence, making errors explicit and recoverable in the agent loop.

3.6.2 Vector search tool (`VectorSearchTool`)

The vector search tool provides local, offline retrieval over a document corpus (`.pdf`, `.docx`, `.txt`, `.md`). Documents are loaded, text is extracted, split into overlapping chunks, indexed with TF–IDF, and ranked by cosine similarity to the query. The tool returns short snippets with metadata (file name, chunk index, similarity score), making retrieval auditable by showing exactly which document region supported a claim.

3.6.3 Web search tool (`DuckDuckGoTool`)

The web search tool retrieves up-to-date information when training-time knowledge is insufficient. It uses DuckDuckGo's `lite`/`html` endpoints and includes practical robustness features (retries, HTTP debug metadata, careful parsing, redirect decoding). It supports simple query modes (e.g., `web:` vs. `news:`) and optional time filtering, and can generate query variants to improve recall. Results are deduplicated and returned as a compact evidence pack (top sources, snippets, timestamps); where useful, it can also extract structured values from snippets or linked pages (e.g., FX rates).

3.6.4 Wikipedia tool (`WikipediaTool`)

The Wikipedia tool targets stable encyclopedic facts as a high-precision alternative to general web search. It uses a layered title-resolution strategy (exact match, suggestions, then ranked search), detects and avoids disambiguation pages, and returns evidence as truncated article text plus structured infobox fields and a source URL. This provides both readable context and machine-friendly facts, improving grounding and supporting later faithfulness analysis.

3.7 My Discoveries during Development

During the implementation and iterative testing, several behaviours were observed that were not obvious from the design phase. These observations emerged from practical experimentation with local models and real tool outputs. As a consequence, they directly influenced both the final architecture and the prompt strategy used in the system.

Schema constraints were more reliable than natural-language instructions

In early versions of the planner prompts, was attempted to enforce consistency of entity naming using natural language instructions. By splitting the query the model was asked to reuse the same wording in all subquestions and avoid synonyms or reinterpretations. In practice, this approach was unreliable. In multi-step plans, the model still introduced synonyms or slightly reformulated entity names, which sometimes propagated errors into later steps.

A noticeable improvement was achieved only after adding an explicit structural anchor in the planning schema, in the form of a `needed_facts` field in the JSON output. Even though this field is no longer useful, its presence in the schema improved naming consistency across steps. The schema effectively acted as a latent constraint that guided the model's internal representation of what information must remain stable across the plan.

This suggests that for structured generation tasks, schema-level constraints can be significantly stronger than natural-language instructions alone. In other words, when deterministic behaviour is required, it is often more effective to encode constraints into the output structure than to rely purely on prompt wording.

Action naming affected routing more than action descriptions

Another unexpected finding was the strong influence of action naming on planner behaviour. An action was implemented intended specifically for resolving pronouns and omitted entities, and named it `NEED CONTEXT`. Despite having clear routing guidance through `when_to_pick` and detailed execution instructions, the planner selected this action more frequently than intended.

The behaviour improved substantially after renaming the action to a more explicit and semantically narrow label: `RESOLVE PRONOUNS AND OMITTED ENTITIES`. This change alone reduced over-triggering and improved alignment between planner intent and actual action selection. The result indicates that under long prompts and constrained context windows, models may rely more heavily on short, high-signal tokens such as action names than on longer descriptive text.

This means that action names should be treated as part of the routing interface, not just as labels. Precise and specific naming reduces ambiguity and improves planner stability.

Small models required intermediate compression of tool outputs

When integrating tools such as Wikipedia retrieval, was observed that long tool outputs degraded the quality of subsequent reasoning steps. When given large raw context windows, these models often failed to consistently extract and reuse the most relevant facts in later steps, leading to unstable or partially grounded answers.

To address this, was introduced an intermediate summarisation stage that compresses each tool result into a small set of high-value facts. The summarisation prompt explicitly instructs the model to preserve exact numeric values and units when they are relevant. This effectively transforms raw observations into a compact, structured reasoning context that is easier for smaller models to process reliably.

Empirically, this change improved step-to-step consistency and reduced context overload. It also reduced prompt token usage and made multi-step reasoning more predictable.

4 Assessment

This chapter evaluates whether the proposed local-first ReAct agent can match or outperform two widely used frameworks (LangChain [11] and LlamaIndex[10]) under identical conditions. All three systems were run with the same local model "qwen3:8b" [25] and equivalent toolset: `web_search`, `wikipedia`, `vector_search`, and `calculator`, with the same small document with facts about Austria for vector retrieval.

4.1 Benchmark Design

The benchmark contains 5 different query intents, each repeated 3 times. So, in total there are 15 runs per agent, 45 runs overall):

1. Current EUR to USD exchange rate (web-dependent, time-sensitive).
2. Number of Austrian states (fact lookup).
3. Multi-step numerical question: "How many people live in the capital city of Austria and what percentage is this number compared to the population of Germany? Round the result to one decimal place.".
4. No-context long literary passage: "But I must explain to you how all this mistaken idea of denouncing pleasure and praising pain was born and I will give you a complete account of the system, and expound the actual teachings of the great explorer of the truth, the master-builder of human happiness" generated by Dummy Text Generator [27] (should trigger clarification/safe handling).
5. Brazil population under mocked wrong tool values (fault-injection robustness).

4.2 Results

4.2.1 Correctness

Correctness was assessed for each run separately by using task-specific criteria. For numerical tasks a run was considered correct only if the final result was exact or numerically/semantically equivalent. For multi-component tasks correctness required full coverage of all requested elements (e.g., both the population value and the corresponding percentage). For context-free tasks, a request for clarification or a safe refusal were considered correct behavior, while any fabricated attribution was considered as incorrect. In the error injection task, a run was correct only if the agent explicitly rejected or corrected the simulated incorrect value in the final response.

For the "exchange rate" query correctness was assessed as "response adequacy" not with strict match accuracy because these rate can vary across time and data sources. Therefore, the expected behavior was a direct response in the form of the current exchange rate with an anchored value.

Table 4.1: Correctness by query type (successful runs out of 3)

Query type	**My Agent**	**LangChain**	**LlamaIndex**
EUR→USD (adequacy)	3/3	1/3	3/3
Austria states	3/3	3/3	3/3
Vienna + % vs Germany	2/3	1/3	3/3
No-context passage	3/3	0/3	1/3
Brazil population (mock fault)	3/3	0/3	1/3
Overall	**14/15 (93.3%)**	**5/15 (33.3%)**	**11/15 (73.3%)**

The proposed agent achieved the highest overall correctness.

4.2.2 Faithfulness

Faithfulness was assessed by analysing tool-usage traces and the final answers. Particular attention was paid to testing whether the agent could resist misleading intermediate observations. My agent demonstrated consistently valid behavior: in the error-injection scenario for the Brazil question, it cross-checked sources (after a web search, it looked up Wikipedia, and vice versa) and corrected incorrect values in all three runs. In context-free queries my agent stopped or asked for clarification("The current step is a literary passage. No further action required.").

In contrast, LangChain propagated incorrect values even when it acknowledged that the data might be outdated ("The population of Brazil is approximately **100.233.432 people** according to the latest data from the specified source. However, please note that this figure might be an estimate or from a specific context, as Brazil's population is generally reported to be around **210 million** based on recent global demographic statistics."). And in the context-free question LangChain agent frequently injected bogus and meaningless values into the final answer and didn't show successful recovery (0/3) ("Your quote's reference to 'denouncing pleasure and praising pain' might metaphorically connect to the report's critique of traditional economic metrics (e.g., GDP) and its advocacy for holistic measures of well-being, such as Finland's sustained top ranking.").

LlamaIndex demonstrated intermediate behavior with partial recovery in the Brazil scenario (1/3) ("The current population of Brazil is approximately 213 million as of 2026, according to recent data from Worldometers and Nations Geo. This figure is significantly higher than the outdated Wikipedia entry, which likely wasn't updated for years.") and mixed results in context-free runs, including some unsupported facts (e.g., "Your passage is a paraphrased version of the famous opening line from Charles Dickens' *A Tale of Two Cities*").

Summing up the "stress testing" results for context-free questions and fictitious, un-

realistic facts (a total of 6 runs), the number of no-support/failure cases was 0/6 for my agent, 6/6 for LangChain, and 4/6 for LlamaIndex.

4.2.3 Efficiency

Table 4.2: Efficiency summary over all 15 runs per agent

Agent	Mean latency	Mean iterations	Mean tool calls
My Agent	140.24 s (02:20.24)	2.00	1.73
LangChain	82.29 s (01:22.29)	1.20	1.13
LlamaIndex	45.62 s (00:45.62)	1.27	1.07

The proposed agent is slower and uses more reasoning/tool steps. This is expected from explicit verification and retry behavior. The trade-off is improved reliability and robustness in error-prone settings.

4.2.4 Robustness

Table 4.3: Robustness-focused outcomes

Metric	My Agent	LangChain	LlamaIndex
Mock-value recovery (Brazil)	3/3	0/3	1/3
Safe handling of no-context input	3/3	0/3	1/3

As discussed earlier in the faithfulness analysis, robustness was stress-tested using (i) a fault-injection scenario (mocked incorrect tool output for Brazil's population) and (ii) a no-context literary passage. The proposed agent is substantially more robust to both tool corruption and ambiguous user input, supporting the thesis claim that explicit assessment and refinement logic improve stability. It consistently rejected or corrected the injected value and safely requested clarification—or stopped—when no question or context was provided. In contrast, LangChain failed all robustness runs (0/6), while LlamaIndex recovered only partially (2/6).

4.3 Discussion

The proposed agent achieved the highest correctness, with 93.3%, and the strongest faithfulness. The efficiency of it was lower than both baselines, but mainly because it performs extra verification steps before giving the final answer. And in terms of robustness the agent had the best recovery behavior in both fault-injection and no-context scenarios.

As a result, the agent demonstrates a practical trade-off between reliability and low speed. Even though it is slower, the agent behaves more safely and more consistently, which matches one of the goals of this thesis: to improve reliability for constrained local LLM setups.

5 Conclusions and Future Work

This thesis investigated whether a **from-scratch, local-first ReAct-style agent** can provide reliable, tool-grounded answers under realistic hardware constraints (local open-source LLMs with $\leq$ 8B parameters) while remaining extensible, maintainable, and evaluation-friendly.

The core motivation was the observation that raw LLMs, despite strong language generation ability, suffer from structural limitations such as hallucinations, weak multi-step planning, stale knowledge, and context-window degradation [4, 3, 2, 5]. The ReAct paradigm addresses these limitations by combining reasoning and external actions (tool use) [7], but existing frameworks often prioritise flexibility and ecosystem integration over strict control, traceability, and reproducibility. This thesis therefore explored whether implementing the control loop explicitly could improve reliability in local-model settings.

The main contribution of this work is the design and implementation of a **modular ReAct agent architecture** that integrates:

- Role-based LLM adapters (planner, summariser, refiner, confidence),
- A configurable action catalogue with Action–Handler separation,
- Tool-centric execution with deterministic input refinement,
- Evidence-first synthesis grounded in tool outputs,
- Confidence-gated retry logic based on modular parameter assessors,
- Structured trace recording and evaluation-ready logging.

The system was implemented as the `my-react-agent` Python package and evaluated against LangChain and LlamaIndex under identical conditions using the same model and toolset. The results show that the implemented from scratch agent achieved the highest overall correctness (93.3%) and the strongest robustness in stress scenarios such as tool fault injection and context-free inputs. The agent demonstrated improved faithfulness by consistently grounding answers in tool evidence and rejecting unsupported or corrupted intermediate results.

However, these improvements come with a measurable cost. The proposed agent showed higher latency and more reasoning iterations than baseline frameworks, primarily due to explicit verification, retry policies, and evidence synthesis. This confirms the expected trade-off between reliability and execution speed in tool-augmented reasoning systems.

Beyond numerical results, an important insight of this thesis is that reliability improvements in small-model agents often come demonstrated the importance of **architectural**

control mechanisms, including structured schemas, explicit intermediate summarisation of tool outputs, and carefully designed routing signals (e.g., action naming). These findings suggest that system-level design decisions can significantly improve behaviour even when model capability is fixed.

5.1 Assumptions and Limitations

Although the results are promising, several assumptions and limitations must be acknowledged.

First, the evaluation was performed using a single model and a relatively small benchmark set. While this ensures controlled comparison conditions, it limits generalisability across different model architectures and task domains.

Second, the system is optimised specifically for local models with constrained context windows and weaker instruction-following capabilities. Some design choices, such as tool-result summarisation and strict tool input refinement, may be less necessary for e.g. very large cloud-hosted models.

Third, the architecture prioritises reliability and traceability over latency. The confidence-gated retry mechanism, multi-step evidence synthesis, and verification loops increase execution time. While this trade-off is intentional and justified for safety-critical or reliability-focused applications, it may not be optimal for low-latency scenarios.

Finally, although the architecture is modular, the current implementation still requires manual configuration of prompts, assessors, and action routing. Automatic self-tuning or learning-based optimisation is not yet implemented.

5.2 Future Work

Future work can be divided into two categories: direct extensions of this project and broader research directions.

5.2.1 Project-Level Future Work

Several concrete improvements can be implemented in future versions of the framework.

One important direction is **adaptive confidence policies**. Currently, retry behaviour is threshold-based. Future versions could dynamically adjust thresholds based on task type, tool reliability history, or model uncertainty signals.

Another direction is **learning-based action routing**. Instead of purely prompt-based decision making, historical traces could be used to learn optimal action selection policies for specific domains.

Tool-level improvements are also possible. For example, implementing tool reliability scoring, automatic tool cross-validation strategies, or tool selection policies based on historical success rates could further improve robustness.

Another important direction is improving **conversation memory and context linking across queries**. At the moment, the agent does not always recover the correct

context from previous turns. This can lead to incorrect tool inputs or incomplete answers. A practical next step is to strengthen memory retrieval by reusing the entity extraction mechanism that is already implemented: entities detected in tool outputs and step summaries can be persisted as structured memory (entity → evidence links, timestamps, and source tool). When a new query arrives, the agent can resolve references by searching this entity store first and injecting the most relevant entity-evidence pairs into the planner and refiner context.

From a performance perspective, future work could focus on **latency optimisation**, such as caching intermediate summaries, reducing redundant LLM calls, or introducing lightweight heuristic filters before invoking expensive reasoning steps.

5.2.2 Broader Research Directions

At a broader level, this work highlights several open research directions.

First, the role of **structured generation constraints** deserves further investigation. This thesis observed that schema-level constraints can outperform natural-language instructions for enforcing consistent reasoning behaviour. Future work could explore formal constraint-guided reasoning architectures.

Second, the interaction between **small models and system design** is still underexplored. Many current research efforts focus on scaling models, while this work suggests that architectural scaffolding can significantly compensate for smaller model limitations.

Third, the concept of **evidence-first synthesis** could be formalised into evaluation standards. Measuring not only correctness but also evidential grounding and trace reproducibility could become standard practice in agent benchmarking.

Finally, the long-term research question remains how to best combine reasoning, tool use, and learning from experience into agents that are both reliable and efficient across a wide range of tasks.

5.3 Final Remarks

This thesis demonstrates that building an agent architecture from scratch is not redundant with existing frameworks when the goal is strict control, traceability, and evaluation transparency under constrained local-model settings.

The results suggest that reliability improvements do not necessarily require larger models; they can also be achieved through better system design, explicit control loops, and strong integration between reasoning, tools, and evaluation mechanisms.

The developed system provides a practical foundation for further research and development of local-first, reliable LLM agents, and contributes empirical evidence that architectural design remains a critical factor in applied AI system performance.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, "Language models are few-shot learners," arXiv:2005.14165, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[2] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," arXiv:2311.05232, 2024. [Online]. Available: https://arxiv.org/abs/2311.05232

[3] A. Matarazzo and R. Torlone, "A survey on large language models with some insights on their capabilities and limitations," arXiv:2501.04040, 2025. [Online]. Available: https://arxiv.org/abs/2501.04040

[4] M. A. K. Raiaan, M. S. H. Mukta, K. Fatema, N. M. Fahad, S. Sakib, M. M. J. Mim, J. Ahmad, M. E. Ali, and S. Azam, "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10433480

[5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," arXiv:2307.03172, 2023. [Online]. Available: https://arxiv.org/abs/2307.03172

[6] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," arXiv:2303.18223, 2025. [Online]. Available: https://arxiv.org/abs/2303.18223

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[8] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman, "Webgpt: Browser-assisted question-answering with human feedback," arXiv:2112.09332, 2022. [Online]. Available: https://arxiv.org/abs/2112.09332

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," arXiv:2005.11401, 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[10] LlamaIndex, "Llamaindex - data framework for llm applications," Online documentation. [Online]. Available: https://www.llamaindex.ai/

[11] LangChain, "Agents - docs by langchain," Online documentation. [Online]. Available: https://docs.langchain.com/oss/python/langchain/agents

[12] D. Docs, "Guide to implementing llm agents: React and simple agents dynamiq docs." [Online]. Available: https://docs.getdynamiq.ai/low-code-builder/llm-agents/guide-to-implementing-llm-agents-react-and-simple-agents

[13] Microsoft, "Autogen: A framework for building ai agents and applications," Online documentation. [Online]. Available: https://microsoft.github.io/autogen/stable/

[14] IBM, "Langgraph overview," Online article, accessed: 2026-02-15. [Online]. Available: https://www.ibm.com/think/topics/langgraph

[15] LlamaIndex, "Reactagent - a simple intro with calculator tools." [Online]. Available: https://developers.llamaindex.ai/python/examples/agent/react_agent/

[16] K. Ning, J. Chen, J. Zhang, W. Li, Z. Wang, Y. Feng, W. Zhang, and Z. Zheng, "Defining and detecting the defects of the large language model-based autonomous agents," arXiv:2412.18371, 2024. [Online]. Available: https://arxiv.org/pdf/2412.18371

[17] D. Chopra, Y. Lu, T. Turakhia, R. Giordano, N. Ryder, and H. Zuegner, "Measuring agents in production," arXiv:2512.04123, 2025. [Online]. Available: https://arxiv.org/abs/2512.04123

[18] R. Pedro, D. Castro, N. Santos *et al.*, "From prompt injections to sql injection attacks: How protected is your llm-integrated web application?" arXiv:2308.01990, 2023. [Online]. Available: https://arxiv.org/abs/2308.01990

[19] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100,000+ questions for machine comprehension of text," arXiv:1606.05250, 2016. [Online]. Available: https://arxiv.org/abs/1606.05250

[20] P. Liang, R. Bommasani, T. Lee *et al.*, "Holistic evaluation of language models," *arXiv:2211.09110*, 2023. [Online]. Available: https://arxiv.org/abs/2211.09110

[21] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," arXiv:2005.00661, 2020. [Online]. Available: https://arxiv.org/abs/2005.00661

[22] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "Ragas: Automated evaluation of retrieval augmented generation," arXiv:2309.15217, 2025. [Online]. Available: https://arxiv.org/abs/2309.15217

[23] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of nlp models with checklist," arXiv:2005.04118, 2020. [Online]. Available: https://arxiv.org/abs/2005.04118

[24] K. Goel, N. Rajani, J. Vig *et al.*, "Robustness gym: Unifying the nlp evaluation landscape," arXiv:2101.04840, 2021. [Online]. Available: https://arxiv.org/abs/2101.04840

[25] Ollama, "Ollama models." [Online]. Available: https://ollama.com

[26] Python Software Foundation, "Python 3 documentation," Online documentation. [Online]. Available: https://docs.python.org/3/

[27] Blindtextgenerator, "Dummy text generator," Online, accessed: 2026-02-15. [Online]. Available: https://www.blindtextgenerator.com/lorem-ipsum